

KOMPILÁTORY

V kapitole o automatech jsme se naučili popisovat jazyky pomocí konečných automatů, regulárních výrazů a gramatik. Zatím šlo hlavně o abstraktní modely. Nyní uvidíme jednu z jejich nejdůležitějších praktických aplikací: překlad programovacích jazyků. Regulární výrazy a konečné automaty nám pomohou rozdělit zdrojový kód na jednotlivé symboly, gramatiky potom popíší, jak z nich lze sestavit příkazy a výrazy.

Překladač ovšem nekončí u ověření správného zápisu. Musí také zjistit, zda program dává smysl, převést jej do vhodné vnitřní reprezentace, případně jej optimalizovat a nakonec vytvořit kód, který lze vykonat. V této kapitole si celý postup ukážeme na malém jazyce implementovaném v Pythonu.

1 Úvodem

Kompilátor je program, který překládá program zapsaný ve zdrojovém jazyce do ekvivalentního programu v cílovém jazyce. Vstup obvykle nazýváme *zdrojovým kódem* a výstup *cílovým kódem*.

Cílovým jazykem nemusí být přímo strojový kód konkrétního procesoru. Překladač může vytvářet assembler, bajtkód pro virtuální stroj, jiný programovací jazyk nebo obecnější mezikód. Podstatné je, aby se zachoval význam programu: pro stejné vstupy má cílový program poskytovat stejné výsledky jako program zdrojový.

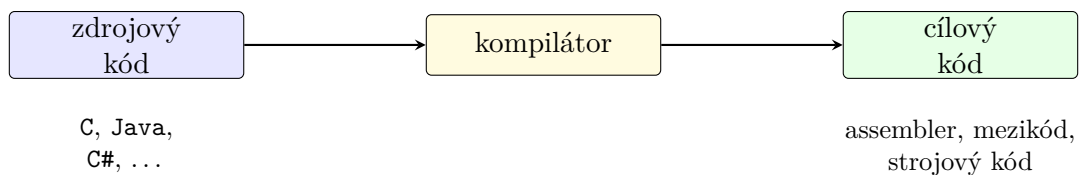


Figure 1: Základní pohled na kompilátor.

1.1 Překladačový řetězec

Slovem *kompilátor* někdy neformálně označujeme celý systém, který vytvoří spustitelný program. Ve skutečnosti může být práce rozdělena mezi několik nástrojů. Typický překlad jednoduchého programu v jazyce C znázorňuje obrázek 2.

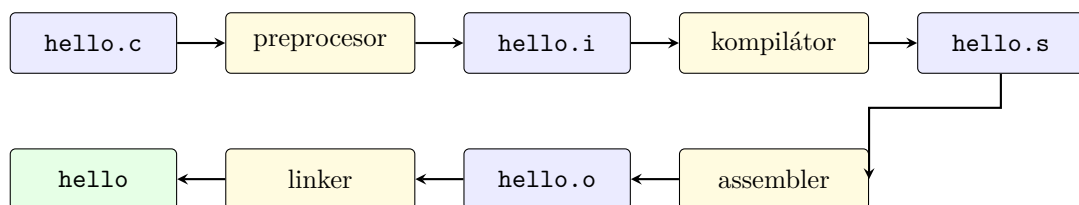


Figure 2: Zjednodušený překladačový řetězec jazyka C.

Jednotlivé části mají následující úlohy:

- *preprocesor* zpracuje direktivy, například vložení hlavičkového souboru pomocí `#include`;

- vlastní *kompilátor* převede předzpracovaný program do assembleru;
- *assembler* vytvoří objektový soubor se strojovými instrukcemi a pomocnými informacemi;
- *linker* neboli sestavovací program spojí objektové soubory a potřebné knihovny do výsledného programu.

Uživatel obvykle spustí jediný ovladač překladače, například `gcc`, který jednotlivé nástroje zavolá za něj. Rozdělení je však důležité při hledání chyb: chyba kompilace, chybějící symbol při linkování a chyba vzniklá až za běhu programu jsou tři různé situace.

Strojový kód a mezikód. *Strojový kód* obsahuje instrukce konkrétního procesoru. Procesor jej může vykonávat přímo, ale stejný soubor zpravidla nelze přenést na počítač s jinou instrukční sadou nebo jiným operačním systémem.

Mezikód je vnitřní reprezentace programu ležící mezi zdrojovým a strojovým kódem. Může být navržen hlavně pro další analýzu a optimalizaci nebo jako přenositelný bajtkód pro virtuální stroj. Díky mezikódu nemusíme pro každou dvojici zdrojového jazyka a cílové platformy vytvářet celý překladač znovu: přední část přeloží různé jazyky do společné reprezentace a koncová část ji převede pro konkrétní počítač.

1.2 Fáze překladače

Překladač zdrojový program obvykle nezpracuje jediným obřím krokem. Postupně jej převádí do reprezentací, se kterými se stále snáze pracuje.

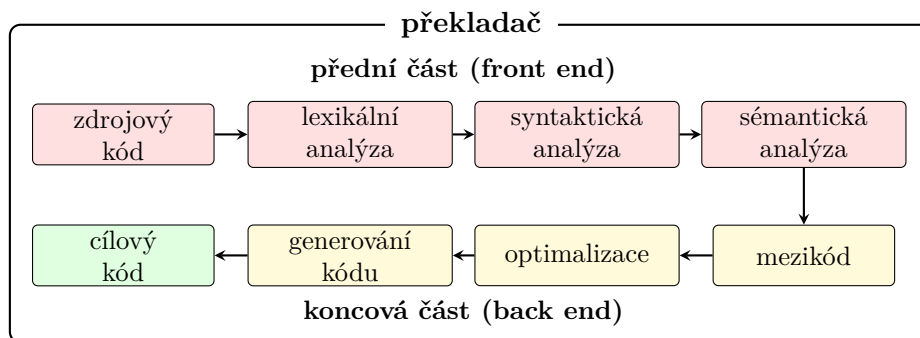


Figure 3: Základní fáze překladače.

1. *Lexikální analýza* seskupí znaky do tokenů.
2. *Syntaktická analýza* z tokenů sestaví strom podle gramatiky.
3. *Sémantická analýza* zkontroluje například deklarace, typy a platnost operací.
4. Z ověřeného stromu vznikne *mezikód*.
5. *Optimalizace* mezikód upraví, aniž by změnila význam programu.
6. *Generování kódu* vytvoří cílový kód.

První tři fáze tvoří *přední část překladače* (anglicky *front end*). Ta závisí hlavně na zdrojovém jazyce. Optimalizace a generování cílového kódu se obvykle řadí do *koncové části* (anglicky *back end*), která více závisí na cílové platformě. Hranice ale není u všech překladačů úplně stejná.

Jednotlivé fáze nemusí odpovídat samostatným programům ani jedinému průchodu zdrojovým textem. Jde především o logické rozdělení odpovědností. Praktický překladač může několik fází spojit nebo některou z nich provést vícekrát.

Průběžný příklad. V dalších sekcích vytvoříme malý jazyk s číselnými výrazy, deklarací proměnné příkazem `let` a výpisem příkazem `print`. Program

```
let x = 2 + 3 * 4;
print x;
```

má vypsát číslo 14. Překladač jej převede do jednoduchého zásobníkového bajtkódu. Ten potom vykoná náš virtuální stroj. Příklad je záměrně malý, ale projde všemi hlavními fázemi z obrázku 3.

2 Lexikální analýza

Zdrojový soubor je pro počítač nejprve jen posloupností znaků. Syntaktický analyzátor by však pracoval velmi nepohodlně, kdyby musel znovu a znovu rozpoznávat, které znaky tvoří jméno proměnné, číslo nebo operátor. Tuto práci proto oddělíme.

Lexikální analyzátor neboli *lexer* čte znaky zdrojového programu a seskupuje je do posloupnosti *tokenů*. Každý token určuje druh rozpoznané části programu a může nést také její hodnotu.

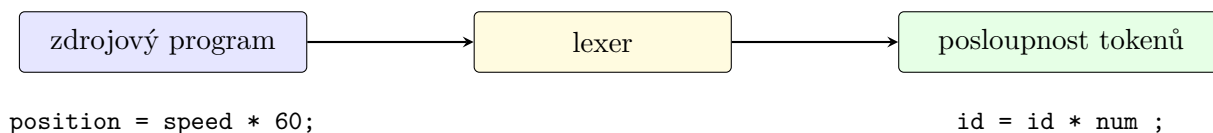


Figure 4: Vstup a výstup lexikálního analyzátoru.

2.1 Lexémy a tokeny

Lexém je konkrétní úsek zdrojového textu. *Token* je informace, kterou o něm lexer předá další fázi. Pro příkaz

```
position = speed * 60;
```

můžeme získat například následující tokeny:

lexém	druh tokenu	předaná hodnota
position	identifikátor	position
=	přiřazení	—
speed	identifikátor	speed
*	násobení	—
60	číslo	60
;	středník	—

U identifikátoru potřebuje parser obvykle znát jeho text, u číselného literálu zase číselnou hodnotu. U operátoru `+` často stačí samotný druh tokenu. Lexer si navíc obvykle ukládá pozici lexému, aby později mohl vypsat srozumitelnou chybovou zprávu.

2.2 Od regulárního výrazu k lexeru

Typické lexémy mají pravidelnou stavbu:

- identifikátor může být popsán zkratkou `[A-Za-z_][A-Za-z0-9_]*`;
- celé nezáporné číslo zápisem `[0-9]+`;
- mezery a konce řádků tvoří jazyk, který lexer rozpozná, ale nepředá parseru;
- každý pevný operátor, například `+` nebo `=`, tvoří jednoprvkový jazyk.

Zápisy v hranatých závorkách a symbol `+` jsou běžné zkratky rozšířených regulárních výrazů: `[0-9]` znamená volbu jedné číslice a `R+` jedno nebo více opakování výrazu `R`. Pomocí základních operací zavedených v sekci o regulárních výrazech je lze rozepsat pouze pomocí sjednocení, zřetězení a Kleeneho hvězdy.

Pro každý druh tokenu můžeme sestavit konečný automat. Jejich počáteční stavy spojíme novým počátečním stavem pomocí λ -přechodů, čímž získáme společný NFA. Pomocí podmnožinové konstrukce jej převedeme na DFA, který pak čte zdrojový text znak po znaku.

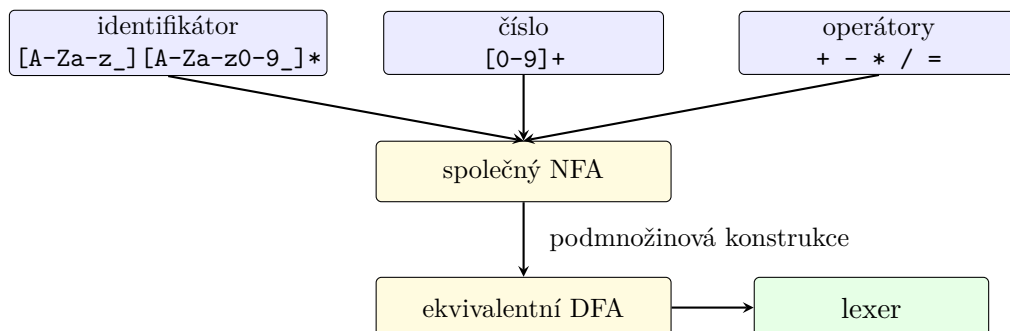


Figure 5: Využití regulárních výrazů a automatů při konstrukci lexeru.

Nejdelší lexém a priorita. Při čtení vstupu může automat projít přijímajícím stavem a později přijmout ještě delší část. Lexer proto obvykle používá pravidlo *nejdelšího lexému*: pokračuje, dokud lze získat delší platný lexém, a vrátí poslední přijatou možnost. Ze vstupu `abc123` tak vznikne jeden identifikátor, nikoliv šest jednopísmenných tokenů.

Pokud stejný lexém vyhovuje více pravidlům, rozhodne jejich priorita. Slovo `let` například splňuje obecný předpis identifikátoru, ale chceme jej rozpoznat jako klíčové slovo. Podobně musí lexer před jedním znakem `=` upřednostnit dvouznakový operátor `==`, pokud jej jazyk obsahuje.

Lexer nerozhoduje, zda tokeny tvoří správný program. Posloupnost `let + ; 7` lze bez problému rozdělit na tokeny, ale její syntaktickou chybu odhalí až parser.

Implementace tokenů. V našem jazyce použijeme dva druhy příkazů a výrazy s identifikátory, čísla a čtyřmi aritmetickými operátory. Potřebujeme také tokeny pro přiřazení, středník a závorky. Jejich druhy a samotný token zachycuje program 2.1.

Program 2.1: Druhy tokenů a datová třída tokenu.

```
1 from dataclasses import dataclass
2 from enum import Enum, auto
3
4
5 class TokenKind(Enum):
6     LET = auto()
7     PRINT = auto()
8     IDENTIFIER = auto()
9     NUMBER = auto()
10    PLUS = auto()
11    MINUS = auto()
12    STAR = auto()
13    SLASH = auto()
14    EQUAL = auto()
15    SEMICOLON = auto()
16    LEFT_PAREN = auto()
17    RIGHT_PAREN = auto()
18    EOF = auto()
19
20
21 KEYWORDS = {
22     "let": TokenKind.LET,
23     "print": TokenKind.PRINT,
24 }
25
26
27 @dataclass(frozen=True)
28 class Token:
29     kind: TokenKind
30     lexeme: str
31     literal: float | None
32     position: int
```

Implementace lexeru. Lexer v programu 2.2 prochází vstup zleva doprava. Čísla a identifikátory čte pomocí samostatných metod, jednoznakové symboly vyhledává ve slovníku. Bílé znaky ignoruje a text od // do konce řádku považuje za komentář.

Program 2.2: Jednoduchý lexikální analyzátor.

```
1 from .tokens import KEYWORDS, Token, TokenKind
2
3
4 class LexerError(ValueError):
5     pass
6
7
8 class Lexer:
9     SINGLE_CHAR_TOKENS = {
10         "+": TokenKind.PLUS,
11         "-": TokenKind.MINUS,
12         "*": TokenKind.STAR,
```

```

13     "/": TokenKind.SLASH,
14     "=": TokenKind.EQUAL,
15     ";": TokenKind.SEMICOLON,
16     "(": TokenKind.LEFT_PAREN,
17     ")": TokenKind.RIGHT_PAREN,
18 }
19
20 def __init__(self, source: str) -> None:
21     self.source = source
22     self.current = 0
23     self.tokens: list[Token] = []
24
25 def scan_tokens(self) -> list[Token]:
26     while not self._at_end():
27         start = self.current
28         char = self._advance()
29
30         if char.isspace():
31             continue
32         if char == "/" and self._peek() == "/":
33             self._skip_comment()
34         elif char.isdigit():
35             self._scan_number(start)
36         elif char.isalpha() or char == "_":
37             self._scan_identifier(start)
38         elif char in self.SINGLE_CHAR_TOKENS:
39             self._add(self.SINGLE_CHAR_TOKENS[char], start)
40         else:
41             raise LexerError(
42                 f"Unexpected character {char!r} at position {start}."
43             )
44
45     self.tokens.append(
46         Token(TokenKind.EOF, "", None, self.current)
47     )
48     return self.tokens
49
50 def _scan_number(self, start: int) -> None:
51     while self._peek().isdigit():
52         self._advance()
53
54     if self._peek() == "." and self._peek_next().isdigit():
55         self._advance()
56         while self._peek().isdigit():
57             self._advance()
58
59     lexeme = self.source[start:self.current]
60     self.tokens.append(
61         Token(TokenKind.NUMBER, lexeme, float(lexeme), start)
62     )
63
64 def _scan_identifier(self, start: int) -> None:
65     while self._peek().isalnum() or self._peek() == "_":
66         self._advance()
67
68     lexeme = self.source[start:self.current]
69     kind = KEYWORDS.get(lexeme, TokenKind.IDENTIFIER)
70     self.tokens.append(Token(kind, lexeme, None, start))

```

```

71
72 def _skip_comment(self) -> None:
73     while not self._at_end() and self._peek() != "\n":
74         self._advance()
75
76 def _add(self, kind: TokenKind, start: int) -> None:
77     lexeme = self.source[start:self.current]
78     self.tokens.append(Token(kind, lexeme, None, start))
79
80 def _advance(self) -> str:
81     char = self.source[self.current]
82     self.current += 1
83     return char
84
85 def _peek(self) -> str:
86     if self._at_end():
87         return "\0"
88     return self.source[self.current]
89
90 def _peek_next(self) -> str:
91     if self.current + 1 >= len(self.source):
92         return "\0"
93     return self.source[self.current + 1]
94
95 def _at_end(self) -> bool:
96     return self.current >= len(self.source)

```

Pro začátek programu `let x = 2 + 3 * 4;` vrátí lexer druhy tokenů

LET, IDENTIFIER, EQUAL, NUMBER, PLUS,
NUMBER, STAR, NUMBER, SEMICOLON, EOF.

Zvláštní token EOF označuje konec vstupu a parseru usnadní rozhodování, zda už přečetl celý program.

3 Syntaktická analýza

Lexer už zdrojový text rozdělil na tokeny. Nyní potřebujeme rozhodnout, zda jsou seřazeny podle pravidel programovacího jazyka, a zachytit jejich strukturu.

Syntaktický analyzátor neboli *parser* dostane posloupnost tokenů a podle gramatiky rozhodne, zda tvoří syntakticky správný program. Jeho výstupem bývá derivační nebo syntaktický strom, případně zpráva o syntaktické chybě.

3.1 Bezkontextové gramatiky

Obecná gramatika zavedená v sekci o gramatikách dovoluje mít na levé straně pravidla celý řetězec. Syntaxi programovacích jazyků však obvykle popisujeme jednodušším druhem gramatik.

Definice 3.1 (Bezkontextová gramatika). Gramatika $G = (V, T, P, S)$ je *bezkontextová*, pokud má každé pravidlo tvar

$$A \rightarrow \alpha,$$

kde $A \in V$ je jediný neterminál a $\alpha \in (V \cup T)^*$.

Název vyjadřuje, že neterminál A smíme přepsat bez ohledu na symboly stojící kolem něj. Regulární gramatiky jsou zvláštním případem bezkontextových gramatik, ale opačná inkluze neplatí. Například pravidlo $S \rightarrow aSb$ je bezkontextové, nikoliv však regulární.

3.2 Aritmetické výrazy

Prioritu běžných aritmetických operátorů zachycuje gramatika

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T, \\ T &\rightarrow T * F \mid T / F \mid F, \\ F &\rightarrow (E) \mid \text{id} \mid \text{num}. \end{aligned}$$

Neterminál E představuje výraz, T term a F faktor. Násobení a dělení se objeví hlouběji ve stromu než sčítání a odčítání, a proto se vyhodnotí dříve. Strom na obrázku 6 odpovídá posloupnosti tokenů

`id + id * num.`

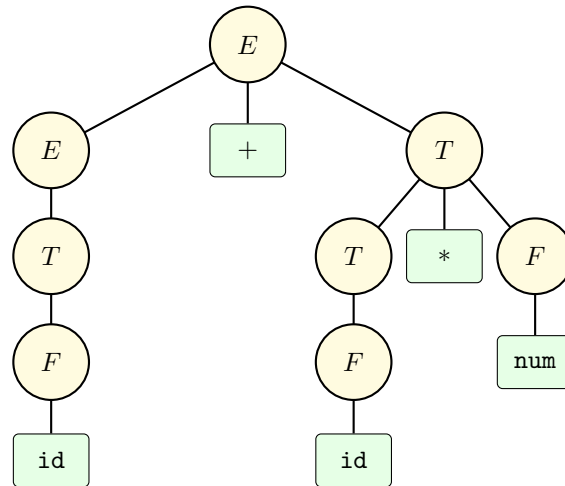


Figure 6: Derivační strom aritmetického výrazu.

Derivační strom, se kterým jsme se poprvé setkali v sekci o gramatikách, obsahuje i neterminály a pomocné kroky gramatiky. Překladač si často vytvoří úspornější *abstraktní syntaktický strom* (zkráceně AST), v němž zůstanou jen informace potřebné pro další zpracování. Ve stromu výrazu $2 + 3 * 4$ bude kořenem sčítání, jeho levým synem číslo 2 a pravým synem násobení čísel 3 a 4.

3.3 Analýza shora dolů a zdola nahoru

Parser *shora dolů* začíná počátečním neterminálem a volí pravidla tak, aby vytvořil vstupní posloupnost tokenů. Strom tedy staví od kořene k listům. Jednoduchým příkladem je rekurzivní sestup, v němž každému neterminálu odpovídá jedna funkce.

Parser *zdola nahoru* naopak začíná tokeny. Hledá úseky odpovídající pravým stranám pravidel a nahrazuje je jejich levou stranou. Strom staví od listů ke kořeni. Praktické generátory parserů často používají některou z variant této metody.

Levá rekurze v pravidle $E \rightarrow E + T$ by pro přímočarý rekurzivní sestup vedla k nekonečnému volání funkce pro E . Pro implementaci proto použijeme ekvivalentní tvar

$$\begin{aligned} E &\rightarrow TE', & E' &\rightarrow +TE' \mid -TE' \mid \lambda, \\ T &\rightarrow FT', & T' &\rightarrow *FT' \mid /FT' \mid \lambda, \\ F &\rightarrow (E) \mid \text{id} \mid \text{num}. \end{aligned}$$

Opakování v E' a T' lze v Pythonu přirozeně zapsat cyklem.

Gramatika malého jazyka. Celý průběžný jazyk popíšeme pravidly

$$\begin{aligned} \text{Program} &\rightarrow \text{Příkaz Program} \mid \lambda, \\ \text{Příkaz} &\rightarrow \text{let id} = E; \mid \text{print } E;, \end{aligned}$$

doplněnými předchozími pravidly pro E , T a F . Jazyk tedy dovoluje deklarovat číselnou proměnnou a vypsat hodnotu výrazu.

Datové třídy v programu 3.1 popisují AST. Společní předkové `Expr` a `Stmt` umožňují snadno rozlišovat výrazy a příkazy.

Program 3.1: Vrcholy abstraktního syntaktického stromu.

```

1 from __future__ import annotations
2
3 from dataclasses import dataclass
4
5 from .tokens import TokenKind
6
7
8 class Expr:
9     pass
10
11
12 @dataclass(frozen=True)
13 class NumberExpr(Expr):
14     value: float
15
16
17 @dataclass(frozen=True)
18 class NameExpr(Expr):
19     name: str
20
21
22 @dataclass(frozen=True)
23 class BinaryExpr(Expr):
24     left: Expr
25     operator: TokenKind
26     right: Expr
27
28
29 class Stmt:
30     pass
31
32
33 @dataclass(frozen=True)
34 class LetStmt(Stmt):
35     name: str

```

```

36     initializer: Expr
37
38
39 @dataclass(frozen=True)
40 class PrintStmt(Stmt):
41     expression: Expr
42
43
44 @dataclass(frozen=True)
45 class Program:
46     statements: list[Stmt]

```

Parser v programu 3.2 přímo odpovídá gramatice. Metoda `_statement` rozpoznává oba druhy příkazů. Metody `_expression` a `_term` zpracovávají operátory, zatímco `_factor` číslo, identifikátor nebo závorku.

Program 3.2: Parser metodou rekurzivního sestupu.

```

1  from .ast_nodes import (
2      BinaryExpr,
3      Expr,
4      LetStmt,
5      NameExpr,
6      NumberExpr,
7      PrintStmt,
8      Program,
9      Stmt,
10 )
11 from .tokens import Token, TokenKind
12
13
14 class ParserError(ValueError):
15     pass
16
17
18 class Parser:
19     def __init__(self, tokens: list[Token]) -> None:
20         self.tokens = tokens
21         self.current = 0
22
23     def parse(self) -> Program:
24         statements: list[Stmt] = []
25         while not self._check(TokenKind.EOF):
26             statements.append(self._statement())
27         return Program(statements)
28
29     def _statement(self) -> Stmt:
30         if self._match(TokenKind.LET):
31             name = self._consume(
32                 TokenKind.IDENTIFIER, "Expected a variable name."
33             )
34             self._consume(TokenKind.EQUAL, "Expected '='.")
35             initializer = self._expression()
36             self._consume(TokenKind.SEMICOLON, "Expected ';'.")
37             return LetStmt(name.lexeme, initializer)
38
39         if self._match(TokenKind.PRINT):
40             expression = self._expression()
41             self._consume(TokenKind.SEMICOLON, "Expected ';'.")

```

```

42         return PrintStmt(expression)
43
44     raise self._error("Expected 'let' or 'print'.")
45
46     def _expression(self) -> Expr:
47         expression = self._term()
48         while self._match(TokenKind.PLUS, TokenKind.MINUS):
49             operator = self._previous().kind
50             right = self._term()
51             expression = BinaryExpr(expression, operator, right)
52         return expression
53
54     def _term(self) -> Expr:
55         expression = self._factor()
56         while self._match(TokenKind.STAR, TokenKind.SLASH):
57             operator = self._previous().kind
58             right = self._factor()
59             expression = BinaryExpr(expression, operator, right)
60         return expression
61
62     def _factor(self) -> Expr:
63         if self._match(TokenKind.NUMBER):
64             literal = self._previous().literal
65             assert literal is not None
66             return NumberExpr(literal)
67
68         if self._match(TokenKind.IDENTIFIER):
69             return NameExpr(self._previous().lexeme)
70
71         if self._match(TokenKind.LEFT_PAREN):
72             expression = self._expression()
73             self._consume(TokenKind.RIGHT_PAREN, "Expected ')'.")
74             return expression
75
76         raise self._error("Expected a number, name, or '('.")
77
78     def _match(self, *kinds: TokenKind) -> bool:
79         for kind in kinds:
80             if self._check(kind):
81                 self.current += 1
82                 return True
83         return False
84
85     def _consume(self, kind: TokenKind, message: str) -> Token:
86         if self._check(kind):
87             self.current += 1
88             return self._previous()
89         raise self._error(message)
90
91     def _check(self, kind: TokenKind) -> bool:
92         return self._peek().kind is kind
93
94     def _peek(self) -> Token:
95         return self.tokens[self.current]
96
97     def _previous(self) -> Token:
98         return self.tokens[self.current - 1]
99

```

```

100     def _error(self, message: str) -> ParserError:
101         token = self._peek()
102         return ParserError(
103             f"{message} At position {token.position}, "
104             f"found {token.lexeme!r}."
105         )

```

Metoda `_consume` zároveň kontroluje povinný následující token. Chybí-li například středník, parser skončí s chybou obsahující pozici neočekávaného tokenu. Kvalitní praktický parser se navíc pokusí najít vhodné místo, od něhož může pokračovat a oznámit více chyb během jednoho překladu.

4 Sémantická analýza

Parser zaručuje, že program odpovídá gramatice. To ještě neznamená, že dává smysl. Příkaz `let x = ;` je syntakticky chybný, protože za rovnítkem chybí výraz. Příkaz `print y;` je naopak syntakticky správný, ale není možné jej vykonat, pokud proměnná `y` nebyla deklarována.

Sémantický analyzátor proto kontroluje podmínky, které nelze nebo není vhodné vyjadřovat samotnou gramatikou. Patří mezi ně zejména existence deklarací, viditelnost jmen, typová správnost výrazů a platnost jednotlivých operací.

4.1 Tabulka symbolů

Překladač si pro každý deklarovaný identifikátor uchovává informace v *tabulce symbolů*. U proměnné to může být jméno, typ, oblast platnosti a místo uložení; u funkce navíc typy parametrů a návratová hodnota.

Při čtení deklarace překladač přidá nový záznam. Při každém použití identifikátoru hledá odpovídající záznam v aktuální a okolních oblastech platnosti. Díky tomu odhalí použití nedeklarované proměnné i nepovolenou opakovanou deklaraci.

4.2 Typová kontrola

Typová kontrola přiřazuje typy listům stromu a postupuje směrem ke kořeni. Je-li například `steps` číslo a `speed` číslo, výsledek jejich násobení je opět číslo. Součet s číselnou proměnnou `position` proto může být přiřazen do další číselné proměnné.

Jiný jazyk může některé kombinace automaticky převádět, například celé číslo na desetinné, zatímco další kombinace odmítne. Pravidla typového systému jsou proto součástí definice konkrétního jazyka, nikoliv univerzální vlastností všech překladačů.

Sémantická kontrola malého jazyka. Náš jazyk má jediný číselný typ. Nemusíme tedy porovnávat různé typy, ale stále musíme zkontrolovat deklarace. Analyzátor v programu 4.1 prochází příkazy v pořadí, ukládá deklarovaná jména do množiny a rekurzivně kontroluje výrazy.

Program 4.1: Sémantická kontrola deklarací a použití proměnných.

```

1 from .ast_nodes import (

```

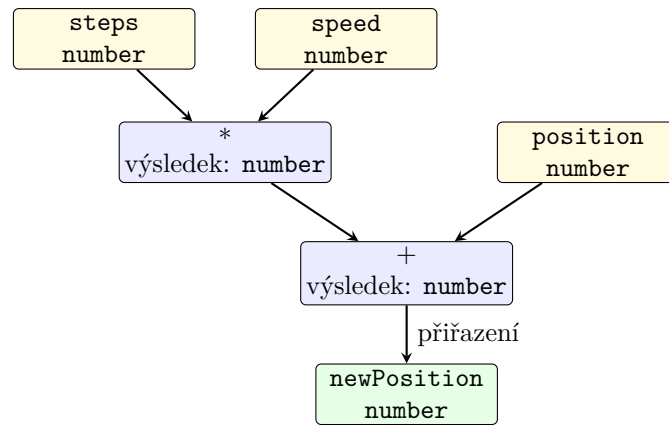


Figure 7: Zjednodušená typová kontrola výrazu.

```

2  BinaryExpr,
3  Expr,
4  LetStmt,
5  NameExpr,
6  NumberExpr,
7  PrintStmt,
8  Program,
9  )
10
11
12 class SemanticError(ValueError):
13     pass
14
15
16 class SemanticAnalyzer:
17     def __init__(self) -> None:
18         self.declared_names: set[str] = set()
19
20     def analyze(self, program: Program) -> None:
21         for statement in program.statements:
22             if isinstance(statement, LetStmt):
23                 if statement.name in self.declared_names:
24                     raise SemanticError(
25                         f"Variable {statement.name!r} is already declared."
26                     )
27                 self._check_expr(statement.initializer)
28                 self.declared_names.add(statement.name)
29             elif isinstance(statement, PrintStmt):
30                 self._check_expr(statement.expression)
31
32     def _check_expr(self, expression: Expr) -> None:
33         if isinstance(expression, NumberExpr):
34             return
35         if isinstance(expression, NameExpr):
36             if expression.name not in self.declared_names:
37                 raise SemanticError(
38                     f"Variable {expression.name!r} is not declared."
39                 )
40             return
41         if isinstance(expression, BinaryExpr):
42             self._check_expr(expression.left)

```

```

43         self._check_expr(expression.right)
44         return
45         raise TypeError(f"Unknown expression: {expression!r}")

```

Inicializační výraz se kontroluje ještě před vložením nového jména do tabulky. Program `let x = x + 1`; proto nemůže použít právě deklarovanou proměnnou, dokud pro ni neexistuje předchozí hodnota. Po úspěšné kontrole v další fázi, že každé jméno odkazuje na existující proměnnou.

Syntaktický strom může být po sémantické analýze doplněn o typy, odkazy do tabulky symbolů a další atributy. Hovoříme pak o anotovaném stromu. Další fáze díky tomu nemusí stejné informace zjišťovat znovu.

5 Optimalizace

Po přední části máme ověřenou reprezentaci programu. Mohli bychom z ní rovnou vytvořit cílový kód, ale výsledek by často prováděl zbytečnou práci.

Optimalizace programu je úprava jeho vnitřní reprezentace, která zachová pozorovatelné chování programu a zlepší zvolené vlastnosti výsledného kódu, například dobu běhu, velikost nebo spotřebu energie.

Slovo „optimalizace“ neznamena, že překladač vždy nalezne nejlepší možný program. Takový požadavek by byl příliš drahý a obecně i algoritmicky problematický. Překladač používá sadu transformací, které bývají výhodné a jejichž správnost umí zdůvodnit.

5.1 Lokální optimalizace

Základní blok je souvislá posloupnost instrukcí, do níž se vstupuje pouze na začátku a z níž se odchází pouze na konci. Uvnitř bloku se tedy tok řízení nevětví. *Lokální optimalizace* pracuje s jedním takovým blokem.

Mezi běžné lokální úpravy patří:

- *výpočet konstant* (anglicky *constant folding*) — výraz $2 \cdot 60$ lze spočítat už při překladu;
- *šíření konstant a kopií* — známe-li $x = 120$, můžeme následující použití x nahradit touto hodnotou;
- *odstranění mrtvého kódu* — výpočet, jehož výsledek se nikde nepoužije, může být zbytečný;
- *odstranění společných podvýrazů* — nezměnily-li se operandy, nemusíme stejný výraz počítat podruhé.

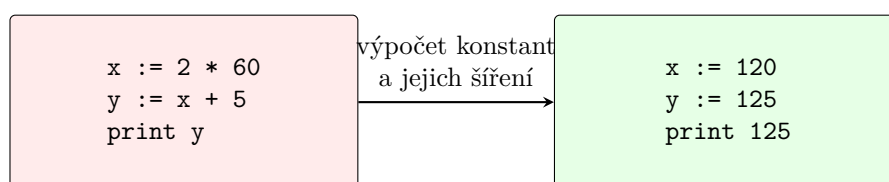


Figure 8: Příklad výpočtu a šíření konstant.

Transformace musí respektovat přesnou sémantiku jazyka. Nahrazení $0 \cdot E$ nulou například není bezpečné, pokud vyhodnocení E může mít vedlejší účinek nebo skončit chybou.

Výpočet konstant v AST. Optimalizátor v programu 5.1 prochází strom zdola nahoru. Jsou-li oba operandy binární operace číselné konstanty, nahradí celý podstrom jediným číselným vrcholem. Dělení nulou záměrně ponechá beze změny, aby příslušná chyba vznikla až při vykonání programu.

Program 5.1: Optimalizace výpočtem konstant.

```
1 from .ast_nodes import (
2     BinaryExpr,
3     Expr,
4     LetStmt,
5     NumberExpr,
6     PrintStmt,
7     Program,
8 )
9 from .tokens import TokenKind
10
11
12 class ConstantFolder:
13     def optimize(self, program: Program) -> Program:
14         statements = []
15         for statement in program.statements:
16             if isinstance(statement, LetStmt):
17                 statements.append(
18                     LetStmt(
19                         statement.name,
20                         self._fold(statement.initializer),
21                     )
22                 )
23             elif isinstance(statement, PrintStmt):
24                 statements.append(
25                     PrintStmt(self._fold(statement.expression))
26                 )
27         return Program(statements)
28
29     def _fold(self, expression: Expr) -> Expr:
30         if not isinstance(expression, BinaryExpr):
31             return expression
32
33         left = self._fold(expression.left)
34         right = self._fold(expression.right)
35
36         if not isinstance(left, NumberExpr):
37             return BinaryExpr(left, expression.operator, right)
38         if not isinstance(right, NumberExpr):
39             return BinaryExpr(left, expression.operator, right)
40
41         a, b = left.value, right.value
42         if expression.operator is TokenKind.PLUS:
43             return NumberExpr(a + b)
44         if expression.operator is TokenKind.MINUS:
45             return NumberExpr(a - b)
46         if expression.operator is TokenKind.STAR:
47             return NumberExpr(a * b)
```

```

48     if expression.operator is TokenKind.SLASH and b != 0:
49         return NumberExpr(a / b)
50     return BinaryExpr(left, expression.operator, right)

```

Ve výrazu $2+3\cdot 4$ se nejprve podstrom $3\cdot 4$ nahradí číslem 12 a potom celý součet číslem 14. Optimalizace tedy přirozeně využívá stromovou strukturu vytvořenou parserem.

5.2 Globální optimalizace a graf toku řízení

Optimalizace přes podmínky, cykly a více bloků potřebuje znát možné pořadí vykonávání programu. Základní bloky proto použijeme jako vrcholy *grafu toku řízení*. Hrana $B_i \rightarrow B_j$ znamená, že po bloku B_i může následovat blok B_j .

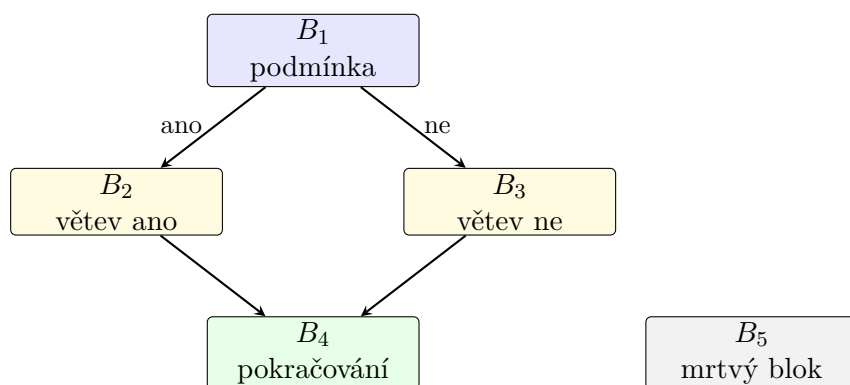


Figure 9: Příklad grafu toku řízení s nedosažitelným blokem B_5 .

Blok je dosažitelný, jestliže do něj vede cesta z počátečního bloku. Nedosažitelné bloky nelze za běhu navštívit a můžeme je odstranit. K jejich nalezení stačí BFS ze sekce o BFS nebo DFS ze sekce o DFS.

Další důležitou informací je *živost proměnné*. Hodnota proměnné je za danou instrukcí živá, pokud může být později přečtena dříve, než ji přepíše jiná hodnota. Instrukce, která zapisuje mrtvou hodnotu a nemá jiný účinek, může být odstraněna.

Ve zjednodušeném základním bloku lze živost spočítat průchodem instrukcí odzadu:

1. začneme množinou proměnných potřebných na konci bloku;
2. proměnnou, do které aktuální instrukce zapisuje, z množiny odstraníme;
3. proměnné, které instrukce čte, do množiny přidáme.

U celého grafu se informace mezi bloky předávají a výpočet se opakuje, dokud se množiny nepřestanou měnit.

5.3 Generování bajtkódu

Po optimalizaci převedeme AST do jednoduchého zásobníkového bajtkódu. Instrukce `PUSH` vloží konstantu na zásobník, `LOAD` načte proměnnou a instrukce `ADD`, `SUB`, `MUL` a `DIV` odeberou dva operandy a vloží výsledek. Příkazy završují instrukce `STORE` a `PRINT`.

Generátor v programu 5.2 prochází výraz v pořadí *levý podstrom*, *pravý podstrom*, *operace*. Tím automaticky vytvoří pořadí vhodné pro zásobníkový stroj.

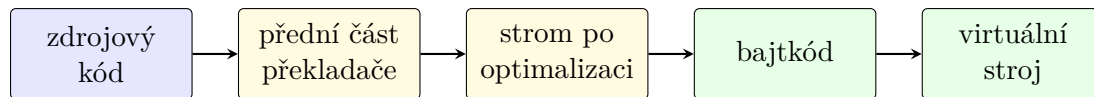


Figure 10: Překlad malého jazyka do bajtkódu pro virtuální stroj.

Program 5.2: Generování jednoduchého zásobníkového bajtkódu.

```

1  from dataclasses import dataclass
2
3  from .ast_nodes import (
4      BinaryExpr,
5      Expr,
6      LetStmt,
7      NameExpr,
8      NumberExpr,
9      PrintStmt,
10     Program,
11 )
12 from .tokens import TokenKind
13
14
15 @dataclass(frozen=True)
16 class Instruction:
17     opcode: str
18     argument: float | str | None = None
19
20
21 class CodeGenerator:
22     OPERATORS = {
23         TokenKind.PLUS: "ADD",
24         TokenKind.MINUS: "SUB",
25         TokenKind.STAR: "MUL",
26         TokenKind.SLASH: "DIV",
27     }
28
29     def generate(self, program: Program) -> list[Instruction]:
30         code: list[Instruction] = []
31         for statement in program.statements:
32             if isinstance(statement, LetStmt):
33                 self._emit_expr(statement.initializer, code)
34                 code.append(Instruction("STORE", statement.name))
35             elif isinstance(statement, PrintStmt):
36                 self._emit_expr(statement.expression, code)
37                 code.append(Instruction("PRINT"))
38             code.append(Instruction("HALT"))
39         return code
40
41     def _emit_expr(
42         self, expression: Expr, code: list[Instruction]
43     ) -> None:
44         if isinstance(expression, NumberExpr):
45             code.append(Instruction("PUSH", expression.value))
46         elif isinstance(expression, NameExpr):
47             code.append(Instruction("LOAD", expression.name))
48         elif isinstance(expression, BinaryExpr):
49             self._emit_expr(expression.left, code)
50             self._emit_expr(expression.right, code)

```

```

51     code.append(Instruction(self.OPERATORS[expression.operator]))
52     else:
53         raise TypeError(f"Unknown expression: {expression!r}")

```

Po optimalizaci se příkaz `let x = 2 + 3 * 4;` přeloží jen na

PUSH 14, STORE x.

Bez optimalizace by vznikly tři instrukce PUSH, násobení, sčítání a teprve potom uložení.

6 Interpretované jazyky

Dosud jsme předpokládali, že překladač vytvoří cílový program a ten se spustí až potom. Druhou základní možností je vykonávat program prostřednictvím jiného programu — interpretu.

Interpret načítá reprezentaci programu a vykonává její příkazy nebo instrukce. Nemusí předem vytvořit samostatný spustitelný soubor se strojovým kódem.

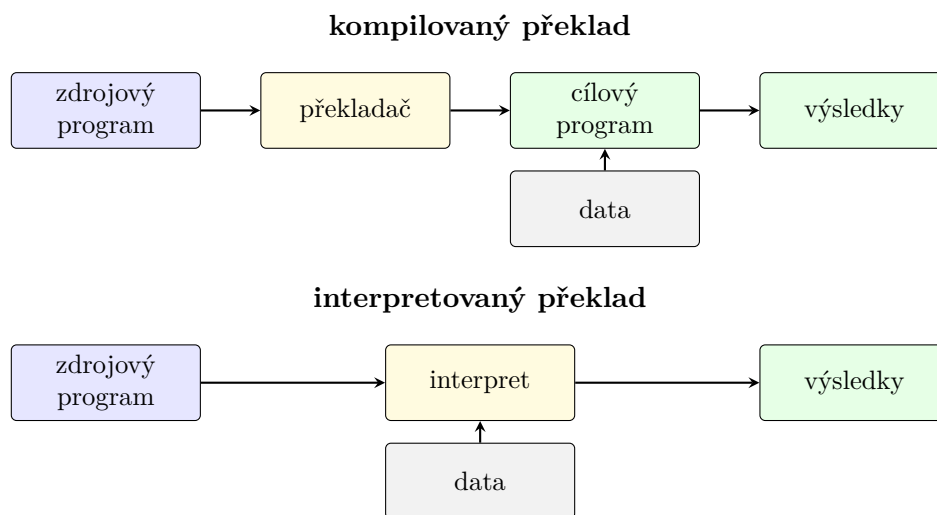


Figure 11: Zjednodušené porovnání kompilovaného a interpretovaného překladu.

Přesnější je mluvit o kompilované nebo interpretované *implementaci*, nikoliv o neměnné vlastnosti jazyka. Tentýž jazyk může mít klasický překladač, interpret i běhové prostředí kombinující více přístupů.

Výhody a nevýhody interpretace. Interpretované vykonávání usnadňuje přenos programu mezi platformami: stačí, aby na nich existoval odpovídající interpret. Často také zjednodušuje interaktivní práci, ladění a rychlé úpravy programu.

Za běhu se však opakuje práce, kterou by klasický překladač mohl provést předem. Interpret musí načítat instrukce své vnitřní reprezentace, rozhodovat o jejich významu a spravovat zásobník či prostředí proměnných. Čistá interpretace proto bývá pomalejší než přímé vykonávání dobře přeloženého strojového kódu. Navíc musí být při spuštění přítomno běhové prostředí.

6.1 Virtuální stroj

Náš překladač vytváří bajtkód. Program 6.1 jej vykonává pomocí zásobníku a slovníku proměnných. Jde tedy o malý *virtuální stroj*: instrukce nejsou instrukcemi skutečného procesoru, ale mají přesně definovaný účinek v našem programu.

Program 6.1: Interpret zásobníkového bajtkódu.

```
1 from collections.abc import Callable
2
3 from .codegen import Instruction
4
5
6 class VirtualMachine:
7     def __init__(
8         self, output: Callable[[float], None] = print
9     ) -> None:
10         self.output = output
11         self.stack: list[float] = []
12         self.variables: dict[str, float] = {}
13
14     def run(self, code: list[Instruction]) -> None:
15         instruction_pointer = 0
16         while True:
17             instruction = code[instruction_pointer]
18             instruction_pointer += 1
19             opcode = instruction.opcode
20
21             if opcode == "HALT":
22                 return
23             if opcode == "PUSH":
24                 assert isinstance(instruction.argument, float)
25                 self.stack.append(instruction.argument)
26             elif opcode == "LOAD":
27                 assert isinstance(instruction.argument, str)
28                 self.stack.append(self.variables[instruction.argument])
29             elif opcode == "STORE":
30                 assert isinstance(instruction.argument, str)
31                 self.variables[instruction.argument] = self.stack.pop()
32             elif opcode == "PRINT":
33                 self.output(self.stack.pop())
34             else:
35                 self._binary_operation(opcode)
36
37     def _binary_operation(self, opcode: str) -> None:
38         right = self.stack.pop()
39         left = self.stack.pop()
40         operations = {
41             "ADD": lambda: left + right,
42             "SUB": lambda: left - right,
43             "MUL": lambda: left * right,
44             "DIV": lambda: left / right,
45         }
46         try:
47             result = operations[opcode]()
48         except KeyError as error:
49             raise ValueError(f"Unknown opcode {opcode!r}.") from error
50         self.stack.append(result)
```

U binární operace leží na vrcholu zásobníku pravý operand a pod ním levý. Virtuální stroj je odebere, spočítá výsledek a ten opět vloží na zásobník. Instrukce `HALT` ukončí interpretační smyčku.

6.2 JIT kompilace

JIT kompilace (z anglického *just-in-time compilation*) překládá mezikód do strojového kódu až za běhu programu. Běhové prostředí může nejprve bajtkód interpretovat, sledovat často používané části a jen ty později přeložit a optimalizovat.



Figure 12: Překlad přes mezikód s možnou JIT kompilací.

Tento přístup kombinuje přenositelnost mezikódu s rychlejším vykonáváním často používaných částí. JIT navíc zná skutečné chování programu za běhu, a může tedy využít informace, které statický překladač neměl. Platí za to časem a pamětí spotřebovanými na překlad během spuštění. Není proto automaticky nejlepší pro každý program. Známými příklady běhových prostředí využívajících JIT jsou virtuální stroj Javy, prostředí .NET a moderní implementace JavaScriptu.

Celý překlad

Program 6.2 spojuje všechny vytvořené části. Zdrojový text projde lexerem, parserem, sémantickou kontrolou, výpočtem konstant a generátorem bajtkódu. Nakonec jej vykoná virtuální stroj.

Program 6.2: Sestavení a spuštění celého malého překladače.

```
1 from .codegen import CodeGenerator
2 from .lexer import Lexer
3 from .optimizer import ConstantFolder
4 from .parser import Parser
5 from .semantic import SemanticAnalyzer
6 from .virtual_machine import VirtualMachine
7
8
9 SOURCE = """
10 let x = 2 + 3 * 4;
11 print x;
12 let y = (x - 2) / 3;
13 print y;
14 """
15
16
17 def main() -> None:
18     tokens = Lexer(SOURCE).scan_tokens()
19     syntax_tree = Parser(tokens).parse()
20     SemanticAnalyzer().analyze(syntax_tree)
21     optimized_tree = ConstantFolder().optimize(syntax_tree)
22     bytecode = CodeGenerator().generate(optimized_tree)
23
```

```

24     for instruction in bytecode:
25         print(instruction)
26
27     print("Program output:")
28     VirtualMachine().run(bytecode)
29
30
31 if __name__ == "__main__":
32     main()

```

Pro přiložený vstup optimalizátor nahradí výraz $2 + 3 \cdot 4$ číslem 14. Vygenerovaný bajtkód nejprve uloží hodnotu do proměnné `x`, vypíše ji, spočítá proměnnou `y` a vypíše také její hodnotu. Výstupem je

```

14.0
4.0

```

Ukázkový překladač je malý, ale hranice mezi částmi odpovídají skutečným systémům: lexer neřeší gramatiku, parser neřeší deklarace, sémantická analýza nemění význam programu a generátor kódu už dostává ověřenou reprezentaci. Právě toto rozdělení umožňuje každou část samostatně testovat a později nahradit propracovanější variantou.